ASSIGNMENT 15.3

V.Sruthi

2303A51308

Batch – 05

TASK – 01

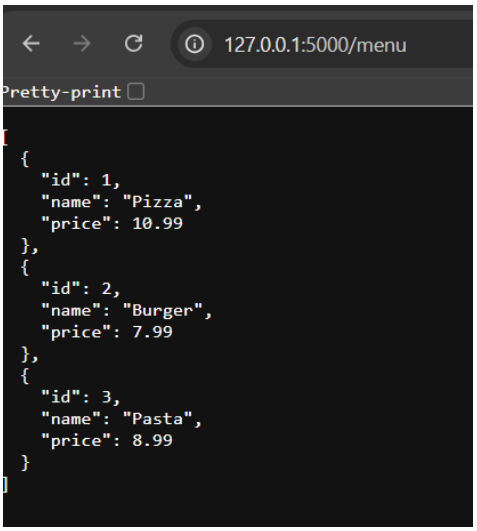
Prompt : Generate a API for online food ordering system in which we can get the menu, place an order,track the order, update existing order and cancel the order.

Code :

```
#Generate a API for online food ordering system in which we
#can get the menu, place an order,track the order, update existing order and cancel the order.
from flask import Flask, request, jsonify
app = Flask(__name__)
# Sample menu
menu = {
    1: {"name": "Pizza", "price": 10},
    2: {"name": "Burger", "price": 5},
    3: {"name": "Pasta", "price": 8}
}
# Sample orders
orders = {}
# Get the menu
@app.route('/menu', methods=['GET'])
def get_menu():
    return jsonify(menu)
# Place an order
@app.route('/order', methods=['POST'])
def place_order():
    order_data = request.get_json()
    order_id = len(orders) + 1
    orders[order_id] = order_data
    return jsonify({"message": "Order placed successfully", "order_id": order_id})
# Track the order
@app.route('/order/<int:order_id>', methods=['GET'])
def track_order(order_id):
    order = orders.get(order_id)
    if order:
        return jsonify(order)
    else:
        return jsonify({"message": "Order not found"}), 404
# Update existing order
@app.route('/order/<int:order_id>',  methods=['PUT'])
```

```python
def update_order(order_id):
    order_data = request.get_json()
    if order_id in orders:
        orders[order_id] = order_data
        return jsonify({"message": "Order updated successfully"})
    else:
        return jsonify({"message": "Order not found"}), 404
# Cancel the order
@app.route('/order/<int:order_id>', methods=['DELETE'])
def cancel_order(order_id):
    if order_id in orders:
        del orders[order_id]
        return jsonify({"message": "Order cancelled successfully"})
    else:
        return jsonify({"message": "Order not found"}), 404
if __name__ == '__main__':
    app.run(debug=True)
# Explanation:
# In this code, we have created a simple API for an online food ordering system using Flask.
# The API has endpoints to get the menu, place an order, track an order, update an existing order, and cancel an order.
# The menu is represented as a dictionary, and orders are stored in another dictionary with unique order IDs.
# Each endpoint handles the respective HTTP methods (GET, POST, PUT, DELETE) to perform the required operations on the orders.
```

Output:



http://127.0.0.1:5000/order

POST http://127.0.0.1:5000/order

Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
  "items": [1, 2]
}
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview JSON

```
{
  "message": "Order placed successfully",
  "order_id": 1
}
```

http://127.0.0.1:5000/order/1

GET http://127.0.0.1:5000/order/1

Params Authorization Headers (7) Body Scripts

Query Params

	Key
	Key

Body Cookies Headers (5) Test Results

Pretty Raw Preview JSON

```
{
    "items": [
        1,
        2
    ]
}
```

http://127.0.0.1:5000/order/1

PUT http://127.0.0.1:5000/order/1

Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw bin

```
1 {
2   "items": [2,3]
3 }
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview JSON

```
1 {
2     "message": "Order updated successfully"
3 }
```

http://127.0.0.1:5000/order/1

DELETE http://127.0.0.1:5000/order/1

Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw b

```
1 {
2   "items": [2,3]
3 }
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview JSON

```
1 {
2     "message": "Order cancelled successfully"
3 }
```

Explanation : This code implements a simple API for an online food ordering system using Flask. It allows users to view the menu, place orders, track orders, update existing orders, and cancel orders. The menu and orders are stored in dictionaries, and the API handles various HTTP methods to perform the necessary operations on the orders.

TASK – 02 :

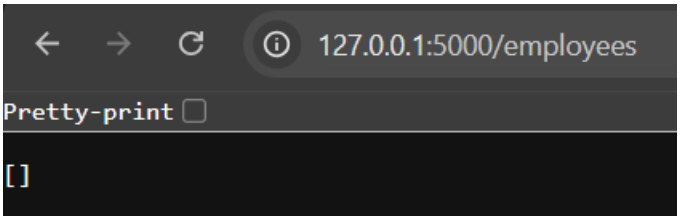
Prompt : Generate a API for Employee Payroll System in which we can list all employees, add a new employee, update an existing employee's salary, and delete an employee.

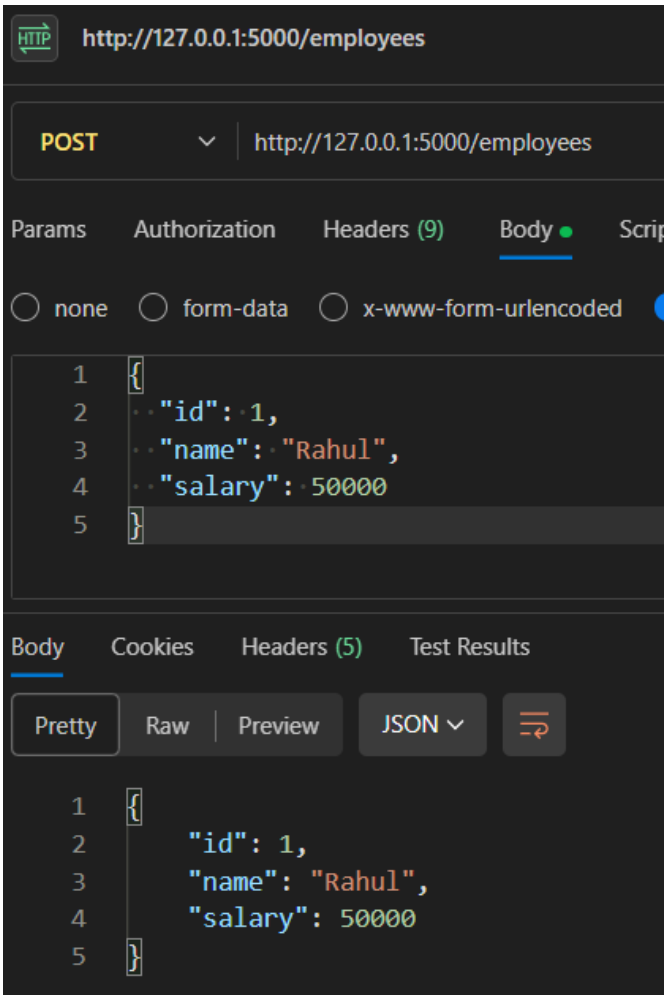
Code :

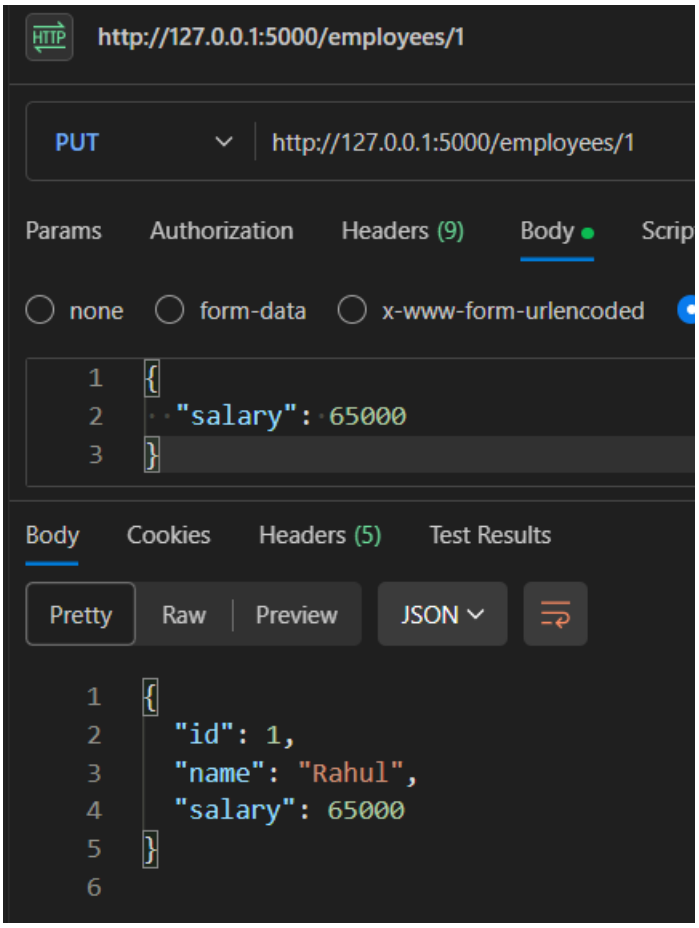
```
#Generate a API for Employee Payroll System in which we can list all
employees, add a new employee, update an existing employee's salary, and
delete an employee.
from flask import Flask, request, jsonify
app = Flask(__name__)
# In-memory database for employees
employees = []
# API to list all employees
@app.route('/employees', methods=['GET'])
def list_employees():
    return jsonify(employees)
# API to add a new employee
@app.route('/employees', methods=['POST'])
def add_employee():
    new_employee = request.get_json()
    employees.append(new_employee)
    return jsonify(new_employee), 201
# API to update an existing employee's salary
@app.route('/employees/<int:employee_id>', methods=['PUT'])
def update_employee_salary(employee_id):
    updated_data = request.get_json()
    for employee in employees:
        if employee['id'] == employee_id:
            employee['salary'] = updated_data['salary']
            return jsonify(employee)
    return jsonify({'error': 'Employee not found'}), 404
# API to delete an employee
@app.route('/employees/<int:employee_id>', methods=['DELETE'])
def delete_employee(employee_id):
    global employees
    employees = [employee for employee in employees if employee['id'] !=
employee_id]
```

```
    return jsonify({'message': 'Employee deleted'})
if __name__== '__main__':
    app.run(debug=True)
# Explanation:
# This code defines a simple API for an Employee Payroll System using Flask.
It includes endpoints to list all employees, add a new employee, update an
existing employee's salary, and delete an employee. The employee data is
stored in an in-memory list for simplicity. Each endpoint handles the
corresponding HTTP method (GET, POST, PUT, DELETE) to perform the required
operations on the employee data.
```

Output :







Explanation : The Employee Payroll API built using Flask manages employee data through REST endpoints.
It allows users to list employees, add new employees, update employee salary, and delete employees using GET, POST, PUT, and DELETE methods.
Employee information is stored in an in-memory list and returned in JSON format.

TASK – 03

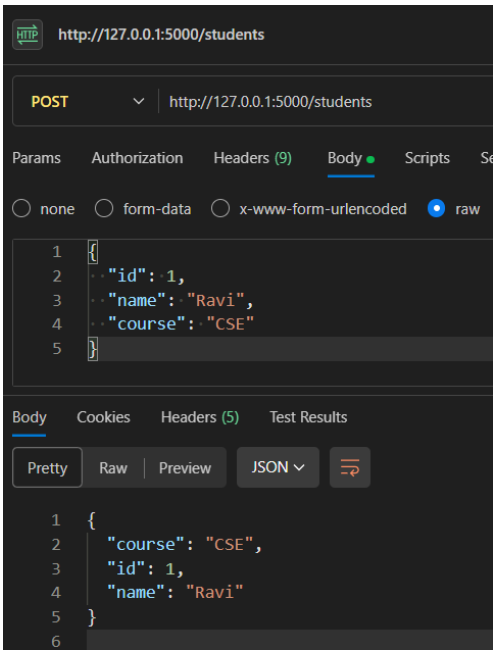
Prompt : Generate a API for Student Information System in which we can list all students, add a new student, update an existing student's information, and delete a student.

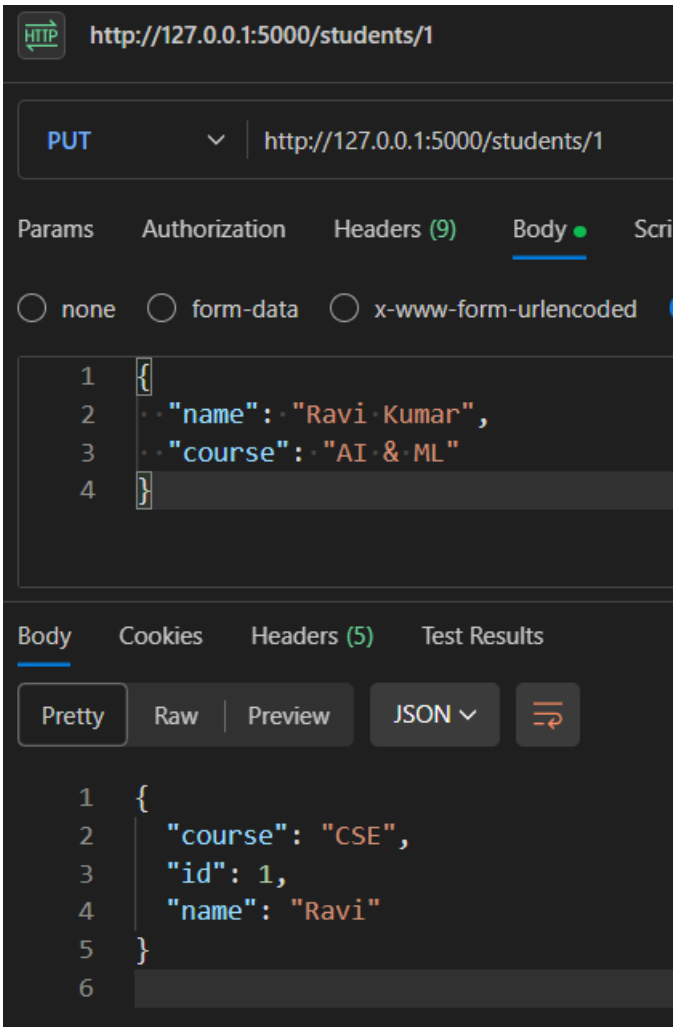
Code :

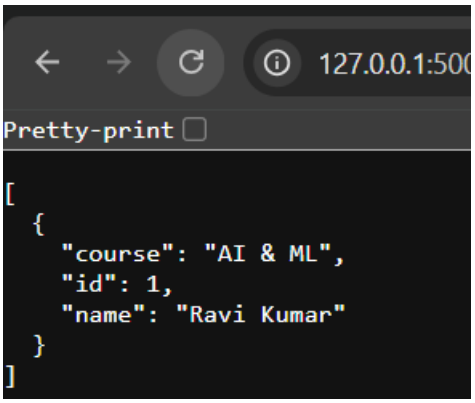
```
#Generate a API for Student Information System in which we can list all
students, add a new student, update an existing student's information, and
delete a student.
from flask import Flask, jsonify, request
app = Flask(__name__)
students = []
```

```python
@app.route('/students', methods=['GET'])
def get_students():
    return jsonify(students)
@app.route('/students', methods=['POST'])
def add_student():
    new_student = request.get_json()
    students.append(new_student)
    return jsonify(new_student), 201
@app.route('/students/<int:id>', methods=['PUT'])
def update_student(id):
    updated_student = request.get_json()
    for student in students:
        if student['id'] == id:
            student.update(updated_student)
            return jsonify(student)
    return jsonify({'message': 'Student not found'}), 404
@app.route('/students/<int:id>', methods=['DELETE'])
def delete_student(id):
    for student in students:
        if student['id'] == id:
            students.remove(student)
            return jsonify({'message': 'Student deleted'})
    return jsonify({'message': 'Student not found'}), 404
if __name__== '__main__':
    app.run(debug=True)
```

Output :







Explanation : This code is a simple API for a Student Information System built using the Flask framework. It allows users to perform CRUD (Create, Read, Update, Delete) operations on student data. The API has endpoints to list all students, add a new student, update an existing student's information, and delete a student based on their ID. The student data is stored in a list called `students`.

TASK – 04

Prompt : Create a Python Flask REST API that integrates with the OpenWeatherMap API to fetch weather data. Add endpoints /weather/current/<city> and /weather/forecast/<city> that return JSON with temperature, humidity, and weather condition.

Code :

```
#Create a Python Flask REST API that integrates with the OpenWeatherMap API to
fetch weather data.
#Add endpoints /weather/current/<city> and /weather/forecast/<city> that
return JSON with temperature, humidity, and weather condition.
from flask import Flask, jsonify

app = Flask(__name__)
@app.route('/weather/current/<city>', methods=['GET'])
def get_current_weather(city):
    # Here you would integrate with the OpenWeatherMap API to fetch current
weather data for the specified city
    # For demonstration purposes, we'll return a mock response
    weather_data = {
        "city": city,
        "temperature": 25,
        "humidity": 60,
        "condition": "Sunny"
    }
    return jsonify(weather_data)
@app.route('/weather/forecast/<city>', methods=['GET'])
def get_weather_forecast(city):
    # Here you would integrate with the OpenWeatherMap API to fetch weather
forecast data for the specified city
    # For demonstration purposes, we'll return a mock response
    forecast_data = {
        "city": city,
        "forecast": [
            {"day": "Monday", "temperature": 26, "condition": "Partly
Cloudy"},
            {"day": "Tuesday", "temperature": 24, "condition": "Rainy"},
            {"day": "Wednesday", "temperature": 27, "condition": "Sunny"}
        ]
    }
    return jsonify(forecast_data)
if __name___== '__main__':
    app.run(debug=True)
```

Output :

127.0.0.1:5000/weather/forecast/<city

Pretty-print


```
{
  "city": "<city",
  "forecast": [
    {
      "condition": "Partly Cloudy",
      "day": "Monday",
      "temperature": 26
    },
    {
      "condition": "Rainy",
      "day": "Tuesday",
      "temperature": 24
    },
    {
      "condition": "Sunny",
      "day": "Wednesday",
      "temperature": 27
    }
  ]
}
```


http://127.0.0.1:5000/weather/current/Chennai

GET http://127.0.0.1:5000/weather/current/Chennai

Params Authorization Headers (7) Body Scripts Settings

Body Cookies Headers (5) Test Results

Pretty Raw Preview JSON


```
1 {
2     "city": "Chennai",
3     "condition": "Sunny",
4     "humidity": 60,
5     "temperature": 25
6 }
```

http://127.0.0.1:5000/weather/forecast/Chennai

GET http://127.0.0.1:5000/weather/forecast/Chennai

Params Authorization Headers (7) Body Scripts Settings

Body Cookies Headers (5) Test Results

Pretty Raw Preview JSON


```
{
    "city": "Chennai",
    "forecast": [
        {
            "condition": "Partly Cloudy",
            "day": "Monday",
            "temperature": 26
        },
        {
            "condition": "Rainy",
            "day": "Tuesday",
            "temperature": 24
        },
        {
            "condition": "Sunny",
            "day": "Wednesday",
            "temperature": 27
        }
    ]
}
```


Explanation : This code creates a simple Flask REST API with two endpoints: /weather/current/<city> and /weather/forecast/<city>. The first endpoint returns mock current weather data for the specified city, while the second endpoint returns a mock weather forecast for the next few days. In a real implementation, you would replace the mock data with actual data fetched from the OpenWeatherMap API.

TASK – 05

Prompt : Create a Python **Flask REST API endpoint to delete an item from a list using the DELETE method. The endpoint should be /items/<int:index>. If the index is invalid, return a JSON error message "Item not found" with status

code 404. If successful, remove the item and return a JSON response with "message": "Item deleted" and the deleted item.

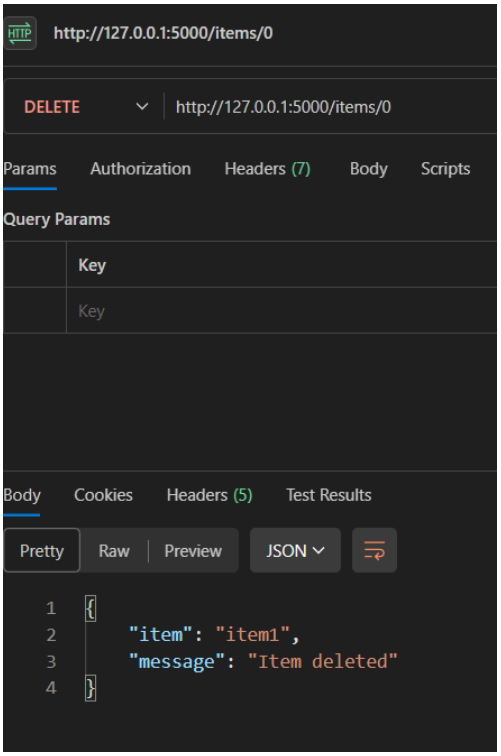
Code :

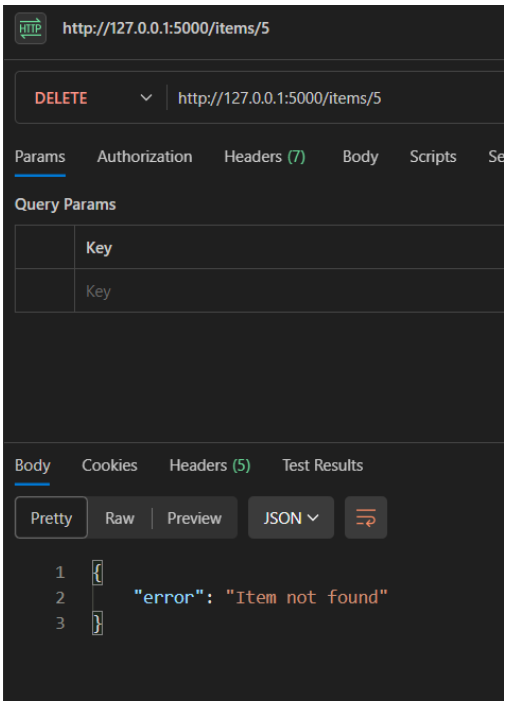
```
#Create a Python **Flask REST API endpoint to delete an item from a list using the DELETE method. The endpoint should be /items/<int:index>. If the index is invalid, return a JSON error message "Item not found" with status code 404. If successful, remove the item and return a JSON response with "message": "Item deleted" and the deleted item.

from flask import Flask, jsonify, request
app = Flask(__name__)
items = ["item1", "item2", "item3"]

@app.route('/items/<int:index>', methods=['DELETE'])
def delete_item(index):
    if 0 <= index < len(items):
        deleted_item = items.pop(index)
        return jsonify({"message": "Item deleted", "item": deleted_item})
    else:
        return jsonify({"error": "Item not found"}), 404
if __name__== '__main__':
    app.run(debug=True)
```

Output :





Explanation :

This code creates a simple Flask REST API with an endpoint to delete an item from a list. The endpoint is defined as `/items/<int:index>`, where `<int:index>` is a placeholder for the index of the item to be deleted.

When a DELETE request is made to this endpoint, the function `delete_item` is called with the index as an argument. The function checks if the index is valid (i.e., within the bounds of the list). If it is valid, it removes the item from the list using `pop()` and returns a JSON response indicating that the item was deleted, along with the deleted item itself.